NAME : KISHORE KUMAR V

REG NO : 192521371

COURSE CODE : ITA 0613

Session 5: K-Nearest Neighbours (K-NN) Algorithm

Session Topic: Write a program for Implementation of K-Nearest Neighbours (K-NN) in Python.

Question 1: K-NN from Scratch for T-shirt Size Prediction

Aim:

To implement the K-Nearest Neighbours algorithm from scratch in Python using Euclidean distance and to predict the T-shirt size of a new customer.

Algorithm:

1. Start. Load the training data (height, weight and T-shirt size) and choose the value of K.
2. Take the new sample whose class is to be predicted.
3. Calculate the Euclidean distance between the new sample and every training sample: $d = \sqrt{((x1 - x2)^2 + (y1 - y2)^2)}$.
4. Sort the distances in ascending order.
5. Select the first K training samples (the K nearest neighbours).
6. Count the class labels of these K neighbours.
7. Assign the class with the majority vote to the new sample and display it.

8. Stop.

Data Set:

T-shirt size data set (12 records):

Height (cm)	Weight (kg)	T-shirt Size
158	58	M
158	59	M
158	63	M
160	59	M
160	60	M
163	60	M
163	61	L
165	64	L
168	63	L
168	66	L
170	68	L
170	72	L

Python Code:

```
import math
from collections import Counter

# T-shirt size data: (height cm, weight kg) -> size
data = [(158, 58, "M"), (158, 59, "M"), (158, 63, "M"),
(160, 59, "M"),
        (160, 60, "M"), (163, 60, "M"), (163, 61, "L"), (165,
64, "L"),
        (168, 63, "L"), (168, 66, "L"), (170, 68, "L"), (170, 72,
"L")]

def euclidean(a, b):
    return math.sqrt((a[0] - b[0]) ** 2 + (a[1] - b[1]) ** 2)

def knn(train, query, k):
    dist = sorted((euclidean(query, (h, w)), s) for h, w, s in
train)
    print(f"{k} nearest neighbours:")
    for d, s in dist[:k]:
        print(f"   distance = {d:.3f}    size = {s}")
```

```
    return Counter(s for _, s in
dist[:k]).most_common(1)[0][0]


query = (161, 61)
k = 5
print("New sample (height, weight):", query)
print("Predicted size:", knn(data, query, k))
```

Output:

```
New sample (height, weight): (161, 61)
5 nearest neighbours:
   distance = 1.414   size = M
   distance = 2.000   size = L
   distance = 2.236   size = M
   distance = 2.236   size = M
   distance = 3.606   size = M
Predicted size: M
```

Result:

The K-NN algorithm was implemented successfully with K = 5. Four of the five nearest neighbours of the new sample

(161 cm, 61 kg) belong to class M, so the predicted size is M.

Question 2: K-NN Classification of the Iris Data Set

Aim:

To implement the K-Nearest Neighbours algorithm using the scikit-learn library and to classify iris flowers into three species.

Algorithm:

1. Start. Import the required libraries and load the Iris data set.
2. Split the data set into training (70%) and testing (30%) sets.
3. Create the K-NN classifier with K = 5.
4. Train (fit) the classifier using the training data.
5. Predict the class labels of the test data.
6. Calculate the accuracy by comparing the actual and predicted labels.
7. Classify a new sample flower and display the result.
8. Stop.

Data Set:

Iris data set (150 records, 10 sample records shown):

Sepal Length	Sepal Width	Petal Length	Petal Width	Class
5.1	3.5	1.4	0.2	setosa
4.9	3	1.4	0.2	setosa
4.7	3.2	1.3	0.2	setosa
7	3.2	4.7	1.4	versicolor
6.4	3.2	4.5	1.5	versicolor
6.9	3.1	4.9	1.5	versicolor
6.3	3.3	6	2.5	virginica
5.8	2.7	5.1	1.9	virginica
7.1	3	5.9	2.1	virginica
5.9	3	5.1	1.8	virginica

Python Code:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
```

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()
Xtr, Xte, ytr, yte = train_test_split(iris.data, iris.target,
test_size=0.3,
                                      random_state=42,
stratify=iris.target)
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(Xtr, ytr)

pred = knn.predict(Xte)
print("Value of K = 5")
print("Accuracy = %.2f%%" % (100 * accuracy_score(yte,
pred)))

for i in range(5):
    print("Test sample", i + 1, "-> Actual:",
iris.target_names[yte[i]],
          "| Predicted:", iris.target_names[pred[i]])
```

```
new = [[6.3, 2.8, 5.1, 1.5]]
print("\nNew sample", new[0], "->",
iris.target_names[knn.predict(new)[0]])
```

Output:

```
Value of K = 5
Accuracy = 97.78%
Test sample 1 -> Actual: virginica | Predicted: virginica
Test sample 2 -> Actual: versicolor | Predicted: versicolor
Test sample 3 -> Actual: virginica | Predicted: virginica
Test sample 4 -> Actual: versicolor | Predicted: versicolor
Test sample 5 -> Actual: virginica | Predicted: virginica

New sample [6.3, 2.8, 5.1, 1.5] -> virginica
```

Result:

The K-NN classifier with K = 5 achieved an accuracy of 97.78% on the Iris test data. The new sample [6.3, 2.8, 5.1, 1.5] was classified as virginica.

Question 3: Finding the Best Value of K for the Wine Data Set

Aim:

To apply the K-NN algorithm on the Wine data set after feature scaling and to find the value of K that gives the highest accuracy.

Algorithm:

1. Start. Load the Wine data set and split it into training (70%) and testing (30%) sets.
2. Standardise the features using StandardScaler, because K-NN depends on distances.
3. Select a range of odd values of K from 1 to 15.
4. For each K, train the K-NN classifier on the training data.
5. Predict the test data and calculate the accuracy for that K.
6. Compare the accuracies and keep the K with the highest accuracy.
7. Display the accuracy table and the best value of K.

8. Stop.

Data Set:

Wine data set (178 records, 13 features, 3 classes; 8 sample records with 4 features shown):

Alcohol	Malic Acid	Ash	Magnesium	Class
14.23	1.71	2.43	127	class_0
13.2	1.78	2.14	100	class_0
13.16	2.36	2.67	101	class_0
12.37	0.94	1.36	88	class_1
12.33	1.1	2.28	101	class_1
12.64	1.36	2.02	100	class_1
12.86	1.35	2.32	122	class_2
12.88	2.99	2.4	104	class_2

Python Code:

```
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

wine = load_wine()
Xtr, Xte, ytr, yte = train_test_split(wine.data, wine.target,
test_size=0.3,
                                      random_state=7,
stratify=wine.target)
sc = StandardScaler().fit(Xtr)            # KNN needs scaled
features
Xtr, Xte = sc.transform(Xtr), sc.transform(Xte)

best_k, best_acc = 1, 0
print("K    Accuracy")
for k in range(1, 16, 2):
    model = KNeighborsClassifier(n_neighbors=k).fit(Xtr,
ytr)
    acc = accuracy_score(yte, model.predict(Xte))
    print(f"{k:<4d} {100 * acc:.2f}%")
    if acc > best_acc:
```

```
        best_k, best_acc = k, acc

print(f"\nBest K = {best_k} with accuracy = {100 *
best_acc:.2f}%")
```

Output:

K Accuracy

1 98.15%

3 98.15%

5 96.30%

7 92.59%

9 92.59%

11 92.59%

13 92.59%

15 92.59%

Best K = 1 with accuracy = 98.15%

Result:

K-NN was tested for K = 1 to 15. The best accuracy of 98.15% was obtained for K = 1, and the accuracy decreased to 92.59% for larger values of K.

Question 4: K-NN Classification of the Breast Cancer Data Set

Aim:

To classify tumours as malignant or benign using the K-NN algorithm and to evaluate the model using accuracy, confusion matrix, precision and recall.

Algorithm:

1. Start. Load the Breast Cancer data set and split it into training (75%) and testing (25%) sets.
2. Standardise the 30 input features.
3. Create the K-NN classifier with K = 5 and fit it on the training data.
4. Predict the class labels of the test data.
5. Calculate the accuracy of the model.
6. Build the confusion matrix from the actual and predicted labels.
7. Calculate the precision and recall and display the results.
8. Stop.

Data Set:

Breast Cancer data set (569 records, 30 features; first 8 records with 4 features shown):

Mean Radius	Mean Texture	Mean Perimeter	Mean Area	Class
17.99	10.38	122.8	1001	malignant
20.57	17.77	132.9	1326	malignant
19.69	21.25	130	1203	malignant
11.42	20.38	77.58	386.1	malignant
20.29	14.34	135.1	1297	malignant
12.45	15.7	82.57	477.1	malignant
18.25	19.98	119.6	1040	malignant
13.71	20.83	90.2	577.9	malignant

Python Code:

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score,
confusion_matrix, precision_score, recall_score

data = load_breast_cancer()
Xtr, Xte, ytr, yte = train_test_split(data.data, data.target,
test_size=0.25,
                                      random_state=1,
stratify=data.target)
sc = StandardScaler().fit(Xtr)
Xtr, Xte = sc.transform(Xtr), sc.transform(Xte)

knn = KNeighborsClassifier(n_neighbors=5).fit(Xtr, ytr)
pred = knn.predict(Xte)

print("Accuracy = %.2f%%" % (100 * accuracy_score(yte,
pred)))
print("\nConfusion Matrix:")
print(confusion_matrix(yte, pred))
print("\nPrecision = %.2f" % precision_score(yte, pred))
print("Recall    = %.2f" % recall_score(yte, pred))
```

Output:

Accuracy = 96.50%

Confusion Matrix:

[[48 5]

 [0 90]]

Precision = 0.95

Recall   = 1.00

Result:

The K-NN model achieved an accuracy of 96.50% on the test data. The confusion matrix shows 138 of 143 samples classified correctly, with a precision of 0.95 and a recall of 1.00.

Question 5: K-NN with Normalisation for Customer Purchase Prediction

Aim:

To implement K-NN from scratch with min-max normalisation to predict whether a customer will buy a product based on age and salary, and to find the leave-one-out accuracy.

Algorithm:

1. Start. Load the customer data (age, salary and purchase decision).
2. Normalise the features using min-max scaling so that salary does not dominate age.
3. Define a function that finds the K nearest neighbours of a sample using Euclidean distance.
4. Predict the class of a sample by the majority class among its K = 3 neighbours.
5. Find the leave-one-out accuracy: leave out each record, predict it from the rest and count the correct predictions.

6. Normalise the new customer and find its 3 nearest neighbours.
7. Display the neighbours and the predicted decision.
8. Stop.

Data Set:

Customer data set (12 records):

Age	**Salary (Rs.)**	**Buys (1 = Yes, 0 = No)**
19	19000	0
25	32000	0
28	41000	0
32	50000	0
35	38000	0
38	72000	1
41	65000	0
44	90000	1
47	58000	0
50	110000	1

Age	Salary (Rs.)	Buys (1 = Yes, 0 = No)
23	27000	0
52	83000	1

Python Code:

```
import numpy as np

# Customer data: [age, salary] -> buys product (1 = Yes, 0 = No)
X = np.array([[19, 19000], [25, 32000], [28, 41000], [32, 50000], [35, 38000],
              [38, 72000], [41, 65000], [44, 90000], [47, 58000], [50, 110000],
              [23, 27000], [52, 83000]], float)
y = np.array([0, 0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1])

mn, mx = X.min(axis=0), X.max(axis=0)
Xn = (X - mn) / (mx - mn)                    # min-max normalisation
```

```
def predict(q, Xtrain, ytrain, k=3):
    d = np.sqrt(((Xtrain - q) ** 2).sum(axis=1))
    idx = np.argsort(d)[:k]
    return np.bincount(ytrain[idx]).argmax(), idx, d[idx]

# Leave-one-out accuracy
hits = 0
for i in range(len(Xn)):
    keep = np.arange(len(Xn)) != i
    p, _, _ = predict(Xn[i], Xn[keep], y[keep])
    hits += (p == y[i])
print("Leave-one-out accuracy = %.2f%%" % (100 * hits / len(Xn)))

new = np.array([42, 80000], float)
p, idx, d = predict((new - mn) / (mx - mn), Xn, y)
print("\nNew customer (age, salary):", tuple(int(v) for v in new))
for i, dist in zip(idx, d):
```

```
    print(f"   neighbour record {i + 1}: distance = {dist:.4f},
class = {y[i]}")
print("Prediction:", "Will buy" if p == 1 else "Will not
buy")
```

Output:

```
Leave-one-out accuracy = 75.00%


New customer (age, salary): (42, 80000)
   neighbour record 8: distance = 0.1255, class = 1
   neighbour record 6: distance = 0.1497, class = 1
   neighbour record 7: distance = 0.1676, class = 0
Prediction: Will buy
```

Result:

The leave-one-out accuracy was 75.00%. For the new customer (age 42, salary 80000), two of the three nearest neighbours were buyers, so the prediction is that the customer will buy.